

15-122: Principles of Imperative Computation — Spring 2025

Practice Problems 11

Due on [Gradescope](#): Sunday 13th April, 2025 by 6pm EDT

Name: _____

Andrew ID: _____

These problems provide practice on the contents of lectures [20 \(virtual machines\)](#), [21 \(graph representation\)](#) and [22 \(graph search\)](#). They will help you prepare for the next check-in.

Preparing your Submission The PDF you submit should have the same number of pages in the same order and with the same margins as the writeup you downloaded.

You can prepare your submission with any PDF editor that you like. Here are a few that prior-semester students recommended:

- On an iPad, use the free **Files** or **Notes** app, or install [Goodnotes](#) or [Notability](#).
 - Don't use the highlighter tool to mark your answers: it shows up poorly when grading.
- [Kami](#), [Adobe Acrobat Online](#), or [DocHub](#), some web-based PDF editors that work from anywhere.
- *Acrobat Pro*, installed on all [non-CS cluster machines](#), works on many platforms.
- [iAnnotate](#) works on any iOS and Android mobile device.

There are many more — use whatever works best for you. If you'd rather not edit a PDF, you can always print this homework, write your answers *neatly* by hand, and scan it into a PDF file — *we don't recommend this option, though*.

Please do not add, remove or reorder pages.

Caution Recent versions of Preview on Mac are buggy: annotations get occasionally deleted for no reason. **Do not use Preview as a PDF editor.**

Submitting your Work Once you are done, submit this assignment on [Gradescope](#). *Always check it was correctly uploaded. You have unlimited submissions.*

Contents

C0VM	1
Graph Union (2.5 points)	3

1 C0VM

This question covers material through lecture 20 (c0vm)

This question deals with the C0VM. The specifications of a selection of C0VM instructions can be found on page 6 of this writeup.

1pt

Task 1.1 Here is a short C0 program that returns the result of an expression:

```
int main() {
    return 112 * 122 - 213 * 100;
}
```

Complete the following bytecode so that it computes the same result as the above program. Write the instructions (e.g., `bipush 122`). The bytecode values (e.g., `10 7A`) are optional. Your bytecode will *not necessarily* be the code generated by `coin -b` on the above program.

Carefully refer to page 6 for how each bytecode instruction works. You cannot assume the existence of any constants or local variables. Here are the hex values of possibly relevant integers (but feel free to use others):

112 = 0x70

122 = 0x7A

-43 = 0xD5

100 = 0x64

10 70	# bipush 112	# 112
10 7A	# bipush 122	# 122
68	# imul	# 13664 // that's 112*122
_____	# _____	
_____	# _____	
_____	# _____	
_____	# _____	
_____	# _____	
_____	# _____	
_____	# _____	
B0	# return	

The following tasks require you to demonstrate your understanding of reading and understanding bytecode for C0.

0.5pts

Task 1.2 Add instructions in the blank lines so that `invokestatic` successfully calls the function `f` passing in 5 for the first argument and 8 for the second argument. Bytecode values are again optional.

```

_____ # _____
_____ # _____
B8 00 01 # invokestatic 1 # f(5, 8)
B0       # return      #

```

0.5pts

Task 1.3 Add instructions in the blank lines on the right such that this function returns 5.

```

BB 04      # new 4      # alloc(int)
_____ # _____
_____ # _____
_____ # _____
2E         # imload     # *(x)
B0         # return     #

```

0.5pts

Task 1.4 If the following function executes without error, write down the value it will return. If this function causes assertion failures or undefined behavior in the VM, then circle the *first* line that will cause that behavior when executed by the C0VM. If there is an error, write a short explanation of the error.

```

BB 04      # new 4
10 00      # bipush 0
10 FF      # bipush -1
4E         # imstore
2E         # imload
B0         # return

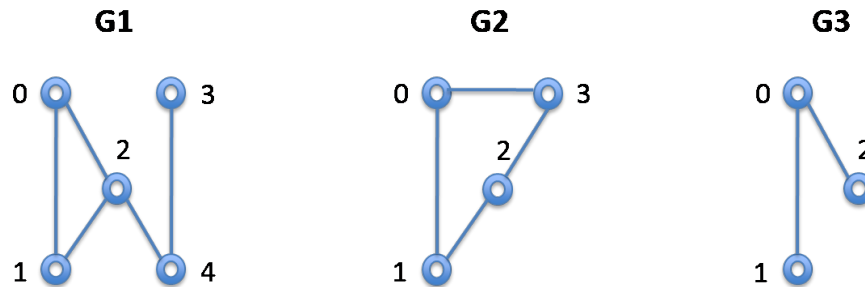
```

2 Graph Union (2.5 points)

This question covers material through lecture 21 (graphs)

The *union* of two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ is defined to be the graph G_{12} whose vertex set is $V_1 \cup V_2$ and whose edge set is $E_1 \cup E_2$ (i.e., all the edges in G_1 and in G_2).

Consider the following graphs G_1 , G_2 and G_3 :



0.5pts

Task 2.1 Draw the graph union of the above graphs G_1 and G_2 in the space below.

0.5pts

Task 2.2 Draw the graph union of the above graphs G_2 and G_3 in the space below.

The code in the remaining tasks uses the graph interface seen in class and on page 7. It also uses the function

```
graph_t graph_copy(graph_t G)
/*@requires G != NULL; @*/
/*@ensures \result != NULL && graph_size(\result) == graph_size(G); @*/ ;
```

which returns a new graph that is identical to G (a deep copy of G).

The function `graph_union` below implements the graph union operation for two graphs as defined above. For simplicity it assumes that the two input graphs have the same set of vertices, that is, $V_1 = V_2$. Note that the function returns a *new* graph.

```

1 graph_t graph_union(graph_t G1, graph_t G2) {
2     REQUIRES(G1 != NULL && G2 != NULL && graph_size(G1) == graph_size(G2));
3
4     graph_t G = graph_copy(G1);
5     for (vertex u = 0; u < graph_size(G); u++) {
6         neighbors_t nbors = graph_get_neighbors(G2, u);
7         while (graph_hasmore_neighbors(nbors)) {
8             vertex w = graph_next_neighbor(nbors);
9             if (!graph_hasedge(G, u, w))
10                graph_addedge(G, u, w);
11        }
12        graph_free_neighbors(nbors);
13    }
14    return G;
15 }
```

Let G_1 and G_2 be two graphs with e_1 and e_2 edges, respectively, and assume that they both have v vertices. We will analyze the cost of graph operations performed on these graphs.

0.5pts

Task 2.3 In the function call `graph_union(G1, G2)`, how many times does the body of the **while** loop on lines 7–11 run **altogether**? Give the simplest and tightest bound.

$O(\rule{1.5cm}{0.4pt})$

0.5pts

Task 2.4 Assume the graph interface is implemented using *adjacency lists* as seen in class. What is the running time of each of the function calls below while calling `graph_union(G1, G2)`? Carefully consider which graph each function is operating on. Recall that G_1 and G_2 have v vertices, and e_1 and e_2 edges respectively. Give the asymptotic complexity for a single call, independently of how many times the loops are run.

`graph_get_neighbors` on line 6: $O(\rule{1.5cm}{0.4pt})$

`graph_hasedge` on line 9: $O(\rule{1.5cm}{0.4pt})$

`graph_addedge` on line 10: $O(\rule{1.5cm}{0.4pt})$

A library-side implementation of the function `graph_copy` has cost $O(v + e)$ — trust us! What is then the overall cost for the call `graph_union(G1, G2)`? (Check one box only.)

☐ $O(e_1 \times e_2)$

☐ $O(e_1 \times e_2 \times v)$

☐ $O(e_1 + v \times e_2)$

☐ $O(e_1 + v^2)$

☐ $O(e_1 + v + e_2 \times \min(v, e_1 + e_2))$

☐ $O(e_1 + e_2 \times \min(v, e_2))$

0.5pts

Task 2.5 If the given graph interface was instead implemented using *adjacency matrices*, the cost of some of the functions would be different. What would be the running time of each of the functions below in this case?

graph_get_neighbors on line 6: $O(\rule{1.5cm}{0.4pt})$

graph_hasedge on line 9: $O(\rule{1.5cm}{0.4pt})$

graph_addege on line 10: $O(\rule{1.5cm}{0.4pt})$

Assume that, for a graph with v vertices and e edges, the graph_copy function has cost $O(v^2)$. What is the overall cost for the function graph_union in this case?

$O(\rule{1.5cm}{0.4pt})$.

REFERENCE: Selected C0VM Instructions

Stack operations

0x59 **dup** $S, v \rightarrow S, v, v$
0x57 **pop** $S, v \rightarrow S$

Arithmetic

0x60 **iadd** $S, x:w32, y:w32 \rightarrow S, x+y:w32$
0x7E **iand** $S, x:w32, y:w32 \rightarrow S, x\&y:w32$
0x6C **idiv** $S, x:w32, y:w32 \rightarrow S, x/y:w32$
0x68 **imul** $S, x:w32, y:w32 \rightarrow S, x*y:w32$
0x80 **ior** $S, x:w32, y:w32 \rightarrow S, x|y:w32$
0x70 **irem** $S, x:w32, y:w32 \rightarrow S, x\%y:w32$
0x78 **ishl** $S, x:w32, y:w32 \rightarrow S, x<<y:w32$
0x7A **ishr** $S, x:w32, y:w32 \rightarrow S, x>>y:w32$
0x64 **isub** $S, x:w32, y:w32 \rightarrow S, x-y:w32$
0x82 **ixor** $S, x:w32, y:w32 \rightarrow S, x\^y:w32$

Local Variables

0x15 **vload** **<i>** $S \rightarrow S, v$ $(v = V[i])$
0x36 **vstore** **<i>** $S, v \rightarrow S$ $(V[i] = v)$

Constants

0x01 **aconst_null** $S \rightarrow S, \text{null}:*$
0x10 **bipush** **** $S \rightarrow S, x:w32$ $(x = (w32)b, \text{sign extended})$
0x13 **ildc** **<c1,c2>** $S \rightarrow S, x:w32$ $(x = \text{int_pool}[(c1<<8)|c2])$
0x14 **aldc** **<c1,c2>** $S \rightarrow S, a:*$ $(a = \&\text{string_pool}[(c1<<8)|c2])$

Control Flow

0x00 **nop** $S \rightarrow S$
0x9F **if_cmpeq** **<o1,o2>** $S, v1, v2 \rightarrow S$ $(pc = pc+(o1<<8|o2) \text{ if } v1 == v2)$
0xA0 **if_cmpne** **<o1,o2>** $S, v1, v2 \rightarrow S$ $(pc = pc+(o1<<8|o2) \text{ if } v1 != v2)$
0xA1 **if_icmplt** **<o1,o2>** $S, x:w32, y:w32 \rightarrow S$ $(pc = pc+(o1<<8|o2) \text{ if } x < y)$
0xA7 **goto** **<o1,o2>** $S \rightarrow S$ $(pc = pc+(o1<<8|o2))$
0xB0 **return** $., v \rightarrow .$ $(\text{return } v \text{ to caller})$

Functions

0xB8 **invokestatic** **<c1,c2>** $S, v1, v2, \dots, vn \rightarrow S, v$
 $(\text{function_pool}[c1<<8|c2] \Rightarrow g, g(v1, \dots, vn) = v)$

Memory

0xBB **new** **<s>** $S \rightarrow S, a:*$ $(*a \text{ is now allocated, size } <s>)$
0x2E **imload** $S, a:*, x:w32 \rightarrow S, x:w32$ $(x = *a, a != \text{NULL}, \text{load 4 bytes})$
0x4E **imstore** $S, a:*, x:w32 \rightarrow S$ $(*a = x, a != \text{NULL}, \text{store 4 bytes})$

YOU MAY TEAR THIS PAGE OFF

REFERENCE: Graph Interface

```
typedef unsigned int vertex;
typedef struct graph_header *graph_t;

graph_t graph_new(unsigned int numvert);
//@ensures \result != NULL;

void graph_free(graph_t G);
//@requires G != NULL;

unsigned int graph_size(graph_t G);
//@requires G != NULL;

bool graph_hasedge(graph_t G, vertex v, vertex w);
//@requires G != NULL;
//@requires v < graph_size(G) && w < graph_size(G);

void graph_addedge(graph_t G, vertex v, vertex w);
//@requires G != NULL;
//@requires v < graph_size(G) && w < graph_size(G);
//@requires v != w && !graph_hasedge(G, v, w);

typedef struct neighbor_header *neighbors_t;

neighbors_t graph_get_neighbors(graph_t G, vertex v);
//@requires G != NULL && v < graph_size(G);
//@ensures \result != NULL;

bool graph_hasmore_neighbors(neighbors_t nbors);
//@requires nbors != NULL;

vertex graph_next_neighbor(neighbors_t nbors);
//@requires nbors != NULL;
//@requires graph_hasmore_neighbors(nbors);

void graph_free_neighbors(neighbors_t nbors);
//@requires nbors != NULL;
```

YOU MAY TEAR THIS PAGE OFF